CSC 252 — Test 2 Study Guide

Banner — scope PROVISIONAL. Test 2 scope is **not published**. Practice covers the lectures so far, Decks 01–04: binary numbers and addition, intro to MIPS, logic gates and adders, and MIPS loops, arrays, and bit shifting. That is the canonical label in the inventory and topic map. The **date is confirmed**: Thu Oct 1, 2026, during lecture, 25 minutes.

Canonical maps: `csc252-topic-map.md` · `announced-assessments-inventory-2026-09-24.md` · `site-audit-2026-09-24.md`

1. Header

Field	Value	Status
Course	CSC 252 Computer Organization (FA26), TuTh 9:30–10:45 AM, Gittings 201	Confirmed
Assessment	Test 2	Confirmed (homepage + syllabus)
Date	**Thu Oct 1, 2026**	**Confirmed**
Length / when	25-minute test during lecture; exact clock time **not published**	Confirmed length; clock time NV
Paper format	Test 1 was a hand-written paper scanned into Gradescope (about 6 pages, graded by manual rubric). Test 2 format is **not published**.	Practice recommendation (inferred from Test 1)
Coverage	Decks 01–04	**PROVISIONAL (from lectures so far)**
Deck 05 (ALU)	Linked for the week of Sep 28. Whether it is on Test 2 is **not stated**.	Not claimed as scope
Guide updated	2026-09-24 (America/Phoenix) — Opus review pass	

All times in this guide are **Arizona time (MST, UTC−7, no daylight saving)**.

Nearby deadlines (for energy planning only — not part of this guide)

Item	When (Arizona)	Note
HW2	Tue Sep 29, 7:00 PM	Gradescope; matches the "two days before the test" rule
Asm 2	Wed Sep 30	**Time conflict:** Gradescope/homepage say 7:00 PM, asm2.pdf says 5:00 PM. The context rule is to prefer Gradescope; finishing by 5:00 PM is safe either way.
Sim 3	Wed Oct 7, 7:00 PM	After the test

Sources

Source	Role
`csc252-topic-map.md`, inventory, site audit	Dates, scope labels, deck list
Slides 01–04 (on box)	Topic content
Gradescope format notes `csc252-2026-09-24.md`	Test 1 structure and your own feedback themes
Syllabus `syllabus_252_fall_26.pdf`	Test and homework rules

---

2. Topic checklist (learning order)

Work across **all** IDs. The emphasis notes in §4b come from your own Test 1 and Sim 1 feedback, not from an invented ranking.

Order	Section ID	Topic	Blind ✓	Check ✓	Retake ✓
1	`252-T2-intro`	Test logistics and allowable-instruction list			
2	`252-T2-binary`	Binary numbers, conversions, 2's complement, addition and subtraction			
3	`252-T2-mips-intro`	Introduction to MIPS			
4	`252-T2-gates`	Logic gates and adders			
5	`252-T2-mips-loops`	MIPS loops, arrays, and bit shifting			

---

3. Plain explanations + worked examples

`252-T2-binary` — Binary, conversions, 2's complement (Deck 01)

Positional value. A digit string $d_{k-1}…d_0$ in base b means $\Sigma\, d_i \cdot b^i$. With k digits the largest unsigned value is $b^k - 1$. For example, 8 bits gives $2^8 - 1 = 255$.

Hex and octal. Group bits in fours (hex) or threes (octal), starting from the **right**. Example: `1011 0111` = 0xB7. Grouped in threes it is `10 110 111`, which is octal 267. Both equal 183.

Approximations. These come up in the "conversions and approximations" style of question.

Power	Exact	Approximate
2^{10}	1,024	≈ 1 thousand
2^{20}	1,048,576	≈ 1 million
2^{30}	1,073,741,824	≈ 1 billion
2^{32}	4,294,967,296	≈ 4 billion

To approximate 2^n, split n into a multiple of 10 plus a remainder. For example, $2^{34} = 2^4 \cdot 2^{30} \approx 16$ billion.

2's complement (k bits).

- The range is -2^{k-1} to $2^{k-1} - 1$. For 8 bits that is −128 to 127.
- **Negative or not:** the value is negative exactly when the most significant bit (MSB) is 1. You do not need to convert the number to decide.
- **Negate:** invert every bit, then add 1. Example: +5 = `0101`, inverted is `1010`, plus 1 gives `1011` = −5 (4-bit).
- **Special patterns:** all 1s is −1. `1000…0` is the most negative value and has no positive partner, because negating it gives itself.
- **Sign-extend:** copy the sign bit into the new high bits. For example, 4-bit `1011` (−5) becomes 8-bit `1111 1011` (still −5).

Binary addition. Work right to left. A column that sums to 2 or 3 writes 0 or 1 and carries 1.

```
  0110   (6)
+ 0101   (5)
------
  1011   (11)
```

Two kinds of overflow. They answer different questions.

- **Unsigned overflow** happens when there is a carry out of the MSB. The true sum does not fit in k bits.
- **Signed overflow** happens when the two inputs have the **same sign** but the result has the **opposite sign**. Adding numbers of opposite signs can never overflow.

Binary subtraction. Compute A − B as A + (−B): invert B, add 1, then add.

- In this method, carry-out = 1 means **no borrow** (A ≥ B as unsigned). Carry-out = 0 means a borrow happened.
- Signed overflow in subtraction occurs when A and B have **different** signs and the result's sign differs from A's.

`252-T2-mips-intro` — Intro MIPS (Deck 02)

- Memory is an array of bytes. A word is 4 bytes, so word addresses step by 4.
- MIPS is a RISC design. ALU instructions only operate on registers; only loads and stores touch memory.
- The register file includes `$zero` (always 0), `$t0–$t9` (temporaries), and `$s0–$s7` (saved). Use the names exactly as the slides do.
- **Immediates are 16-bit signed.** This is why MIPS has `addi` but no real `subi`: to subtract a constant, add a negative one (`addi $t0, $t1, -7`). MARS may accept `subi` as a pseudo-instruction, but it is not a real instruction and is usually not on the allowable list.
- `slt rd, rs, rt` sets rd to 1 if rs < rt, else 0. Combined with `beq`/`bne` against `$zero`, it replaces pseudo-branches like `blt`.

Allowable instructions. Test 1 printed an allowable-instruction list on its cover. Practice as if Test 2 does the same: `add`, `addi`, `sub`, `and`, `andi`, `or`, `ori`, `xor`, `nor`, `sll`, `srl`, `sra`, `slt`, `slti`, `lw`, `sw`, `lb`, `sb`, `beq`, `bne`, `j`, `la`, `syscall`. **Check the real cover sheet on test day.** This list is a practice assumption, not a published Test 2 list.

`252-T2-gates` — Gates and adders (Deck 03)

- **Sum of products:** for each truth-table row that outputs 1, AND the inputs together (inverting any that are 0), then OR those terms.
- **Half adder:** Sum = A XOR B; Carry = A AND B.
- **Full adder:** Sum = A XOR B XOR Cin; Cout = (A AND B) OR (A AND Cin) OR (B AND Cin). This is the majority function.
- **Ripple-carry adder:** chain full adders so each Cout feeds the next bit's Cin.

Full adder truth table (complete):

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sum is 1 when an odd number of inputs are 1. Cout is 1 when at least two inputs are 1.

`252-T2-mips-loops` — Loops, arrays, bit shifting (Deck 04)

- **Loop shape:** initialize, test (branch **out** when the condition is false), body, update, then `j` back to the test.
- **Array element i:** address = base + i·4 for words (`sll $t1, $t0, 2` then `add`). For bytes (such as string characters) the address is base + i with `lb`/`sb`.
- **Short-circuit conditions.**
- For `A && B`: if A is false, skip to the else/end immediately.
- For `A || B`: if A is true, jump into the body immediately.
- **Shifts:**
- `sll` by k multiplies by 2^k, provided no 1-bits fall off the top.
- `srl` by k divides an **unsigned** value by 2^k.
- `sra` keeps the sign bit, so it suits signed division (rounding toward $-\infty$).
- **Masks:** `andi` with 0xF keeps the low 4 bits. `ori` sets bits. `xor` with all 1s flips bits.

Worked example — `if (x == y && a < b) body;` with x, y, a, b in `$s0`–`$s3`:

```
      bne  $s0, $s1, SKIP      # x != y  -> whole && is false
      slt  $t0, $s2, $s3       # t0 = (a < b)
      beq  $t0, $zero, SKIP    # a >= b  -> false
      # ... body ...
SKIP:
```

---

4. Practice questions (core)

Attempt these **blind**, on paper, before looking at §5.

P1 (recall / binary): What is the largest unsigned value with 8 bits?

P2 (solve / binary): Convert decimal 43 to 8-bit binary and to hex.

P3 (solve / 2's complement): In 8-bit 2's complement, what is −17? Show invert-then-add-1 starting from +17.

P4 (explain / overflow): When does signed overflow happen for addition? How is that different from unsigned overflow?

P5 (recall / gates): Give the Sum and Carry expressions for a half adder.

P6 (trace / full adder): For A=1, B=0, Cin=1, what are Sum and Cout?

P7 (explain / MIPS intro): Why does MIPS keep load/store separate from ALU instructions?

P8 (code / MIPS): Write a loop that sums N words. The base address is in `$s0`, the count in `$s1`, and the result goes in `$s2`.

P9 (trace / arrays): A word array starts at `0x10010000`. What is the byte address of element i = 3 (0-based)?

P10 (explain / shifts): Why does shifting left by 1 multiply an unsigned value by 2, and when does that fail?

P11 (trace / branches): For `if (x != y || a < b)`, when can the second check be skipped?

P12 (recall / logistics): When is Test 2, and how long is it?

4b. Gradescope-format practice — Practice recommendation

Where this format comes from. It is inferred from the **Test 1 result structure** in the Gradescope format notes. Test 1 was a scanned hand-written paper of about 6 pages with four 25-point questions. Your graded feedback listed these themes:

- Question 1 (short concepts: 2's complement, `&` vs `&&`, "negative or not", binary patterns, SUBI) was the **largest loss**.
- Binary Subtraction and C-to-MIPS also lost points.
- Sim 1 autograder failures clustered on 2's complement and SUB.

The groups below weight those themes. **This is not a published Test 2 blueprint**, and four equal buckets is **not** a confirmed Test 2 format. Topics stay inside provisional Decks 01–04.

Timed mock (recommended once, on Mon Sep 28): on blank paper, do A4, A5, A6, B2, B5, C3, C4, and D4 in **25 minutes**. Write by hand, since Test 1 was scanned paper.

Group A — Short concepts (Test 1 Question 1 themes)

GS-A1. In one sentence each: (i) what does sign extension do when widening a 2's complement value? (ii) how does `&` differ from `&&`?

GS-A2. What extra input does a full adder have compared with a half adder? Why do all bits after the least significant bit need full adders?

GS-A3. If each word is 4 bytes, how do you turn array index `i` into a byte offset in MIPS?

GS-A4 (negative or not). Treat each value as an 8-bit 2's complement pattern. Is it negative? `0x7F`, `0x80`, `0xFF`, `0x3C`.

GS-A5 (SUBI). MIPS has `addi` but no real `subi`. Write one instruction for `$t0 = $t1 - 7`, and explain why MIPS does not need `subi`.

GS-A6 (`&` vs `&&`). Evaluate in C: `6 & 3`, `6 && 3`, `4 & 2`, `4 && 2`. What bug does the last pair illustrate?

GS-A7 (binary patterns). For 8-bit 2's complement, give the bit patterns for (i) the most negative value, (ii) −1, (iii) the largest positive value. Which value has no positive counterpart?

Group B — Conversions and approximations

GS-B1. Convert unsigned decimal 157 to 8-bit binary and to hex. Show the grouping.

GS-B2. Interpret `1101 0110` as (i) unsigned decimal and (ii) 8-bit 2's complement decimal. Show work.

GS-B3. Which power of two is closest to 1000?

GS-B4. About how large is 2^{32}? Estimate without a calculator and say how you split the exponent.

GS-B5. What is the minimum number of bits needed to give 3,000,000 items distinct unsigned IDs?

GS-B6. Convert `0xB7` to binary, to decimal, and to octal.

Group C — Binary arithmetic (subtraction emphasis)

GS-C1. Add unsigned 8-bit `0110 1101` + `0011 1010`. Show carries. Give the sum in binary and decimal.

GS-C2. Add 8-bit 2's complement `0101 1100` + `0011 0101`. Does signed overflow occur? Explain in one sentence.

GS-C3. Subtract unsigned 8-bit `1001 0110` − `0010 1101`. Show borrows, or use 2's complement addition. Give the difference in binary and decimal.

GS-C4. Compute `0x25 – 0x3A` in 8 bits using "invert, add 1, add". Give the result pattern and its signed value. Did a borrow occur, and how does the carry-out tell you?

GS-C5. In 8-bit signed arithmetic, compute 100 − (−50) using bit patterns. Does signed overflow occur? How can you tell from the signs alone?

Group D — C-to-MIPS (allowable-instruction practice)

Use only the practice allowable list from §3. Do not use pseudo-branches such as `blt` or `bge`.

GS-D1. Write `$t0 = ($t1 < $t2) ? 1 : 0` in one instruction.

GS-D2. Trace: `$s0 = 0x10010000`, `$t0 = 2`, then `sll $t1,$t0,2` / `add $t1,$s0,$t1` / `lw $t2,0($t1)`. What byte address is loaded?

GS-D3. Write a 3–4 instruction loop that counts `$t0` down to 0.

GS-D4 (full translation). With `$s0 = arr` (int array), `$s1 = n`, and `$s2 = count`, translate:

```
count = 0;
for (i = 0; i < n; i++)
    if (arr[i] < 0)
        count++;
```

GS-D5 (short-circuit OR). With a, b, c, d, x in `$s0`–`$s4`, translate `if (a == b || c < d) x = 1;`

GS-D6 (bit field). Put bits 4–7 of `$s0` into the low bits of `$t0`, i.e. `$t0 = ($s0 >> 4) & 0xF`.

Hand-in habit (Homework-style) — Practice recommendation

The Gradescope Homework 2 upload dialog asks for **images per question or one PDF**, then asks you to **assign PDF pages to questions**. When you practice on paper, put each question on its own page and label it, so page assignment is fast when you submit real homework. **Do not submit this guide or its answers.**

5. ANSWER KEY (separate — check only after a blind attempt)

Core keys

A1. $2^8 - 1 = 255$.

A2. 43 = 32 + 8 + 2 + 1, so `0010 1011` = `0x2B`.

A3. +17 = `0001 0001`. Inverted: `1110 1110`. Plus 1: `1110 1111` = −17.

A4. Signed overflow happens when two same-sign inputs give an opposite-sign result (two positives giving a negative, or two negatives giving a positive). Unsigned overflow is a carry out of the MSB. One sum can have either kind, both, or neither.

A5. Sum = A XOR B; Carry = A AND B.

A6. Sum = 1 ⊕ 0 ⊕ 1 = 0. Cout = majority(1, 0, 1) = 1.

A7. Keeping ALU instructions register-only, with explicit load/store, keeps instructions simple and uniform. That is the RISC trade-off: simpler, faster hardware.

A8.

```
      add  $s2, $zero, $zero    # sum = 0
      add  $t0, $zero, $zero    # i = 0
LOOP: beq  $t0, $s1, DONE       # i == n -> done
      sll  $t1, $t0, 2          # byte offset = i*4
      add  $t1, $s0, $t1        # &arr[i]
      lw   $t2, 0($t1)
      add  $s2, $s2, $t2
      addi $t0, $t0, 1
      j    LOOP
DONE:
```

A9. Offset = 3 × 4 = 12 = 0xC, so the address is `0x1001000C`.

A10. Each bit moves to the next higher power of 2, and a 0 enters at the bottom, so the value doubles. It fails (overflows) when a 1-bit is shifted out of the top.

A11. If `x != y` is already true, the whole OR is true, so skip the `a < b` test and enter the body.

A12. Thu Oct 1, 2026, during lecture (class meets 9:30–10:45 AM). The test is 25 minutes. The exact clock time within lecture is not published.

Gradescope-format keys

GS-A1. (i) It copies the sign bit into the new high bits, so the value is unchanged. (ii) `&` is bitwise AND on every bit pair. `&&` is logical AND: the result is 0 or 1, and the right side is skipped if the left side is false.

GS-A2. The full adder adds a carry-in (Cin). Every bit after the least significant must add the carry coming from the bit below it.

GS-A3. Byte offset = i × 4, usually `sll $t1, $t0, 2`.

GS-A4. `0x7F` = 127, not negative. `0x80` = −128, negative. `0xFF` = −1, negative. `0x3C` = 60, not negative. Only the MSB matters.

GS-A5. `addi $t0, $t1, -7`. The 16-bit immediate is signed, so adding a negative constant already does subtraction, and a separate opcode is unnecessary.

GS-A6. `6 & 3` = `110 & 011` = `010` = 2. `6 && 3` = 1. `4 & 2` = `100 & 010` = 0. `4 && 2` = 1. The last pair shows the bug: two "true" values can AND bitwise to 0 (false), so using `&` where you meant `&&` changes the program's logic.

GS-A7. (i) `1000 0000` = −128. (ii) `1111 1111`. (iii) `0111 1111` = 127. −128 has no positive counterpart, because +128 does not fit in 8 bits.

GS-B1. 157 = 128 + 16 + 8 + 4 + 1, so `1001 1101` = `0x9D`.

GS-B2. Unsigned: 128 + 64 + 16 + 4 + 2 = 214. Signed: the MSB is 1, so it is negative. Invert to `0010 1001`, add 1 to get `0010 1010` = 42, so the value is **−42**. Check: 214 − 256 = −42.

GS-B3. 2^{10} = 1024.

GS-B4. $2^{32} = 2^2 \cdot 2^{30} \approx 4 \times 1$ billion ≈ 4 billion. The exact value is 4,294,967,296.

GS-B5. 22 bits. $2^{21} \approx 2.1$ million is too small, and $2^{22} \approx 4.2$ million is enough.

GS-B6. `1011 0111` = 183. Grouped in threes: `10 110 111`, octal **267**. Check: 2·64 + 6·8 + 7 = 183.

GS-C1. 109 + 58 = 167 = `1010 0111`. There is no carry out, so no unsigned overflow.

GS-C2. `0x5C + 0x35 = 0x91` = `1001 0001`. **Yes, signed overflow:** both inputs are positive (MSB 0), but the result's MSB is 1 (negative).

GS-C3. 150 − 45 = 105 = `0110 1001`.

GS-C4. −`0x3A`: `0011 1010` inverted is `1100 0101`, plus 1 is `1100 0110` (0xC6). Then `0010 0101 + 1100 0110 = 1110 1011` = `0xEB`. As a signed value that is 235 − 256 = **−21** (check: 37 − 58 = −21). The carry-out is **0**, which means a borrow occurred: as unsigned values, 37 < 58.

GS-C5. −(−50) = +50 = `0011 0010`, and 100 = `0110 0100`. Their sum is `1001 0110` = 0x96, which reads as −106. **Signed overflow:** a positive minus a negative must be positive, but the result is negative. The true answer, 150, exceeds 127.

GS-D1. `slt $t0, $t1, $t2`

GS-D2. 2 × 4 = 8, so the address is `0x10010008`.

GS-D3.

```
LOOP: beq  $t0, $zero, DONE
      addi $t0, $t0, -1
      j    LOOP
DONE:
```

GS-D4.

```
      addi $s2, $zero, 0        # count = 0
      addi $t0, $zero, 0        # i = 0
LOOP: slt  $t1, $t0, $s1        # t1 = (i < n)
      beq  $t1, $zero, DONE     # exit when !(i < n)
      sll  $t2, $t0, 2          # i*4
      add  $t2, $s0, $t2        # &arr[i]
      lw   $t3, 0($t2)          # arr[i]
      slt  $t4, $t3, $zero      # t4 = (arr[i] < 0)
      beq  $t4, $zero, NEXT
      addi $s2, $s2, 1          # count++
NEXT: addi $t0, $t0, 1          # i++
      j    LOOP
DONE:
```

Graders usually check three things: the loop test exits on the **false** condition, the offset is ×4, and `i++` runs on both paths.

GS-D5.

```
      beq  $s0, $s1, THEN       # a == b -> OR already true
      slt  $t0, $s2, $s3        # t0 = (c < d)
      beq  $t0, $zero, END      # both false -> skip
THEN: addi $s4, $zero, 1        # x = 1
END:
```

GS-D6. `srl $t0, $s0, 4` then `andi $t0, $t0, 0xF`. If `andi` is not allowed, use `addi $t1, $zero, 15` then `and $t0, $t0, $t1`.

---

6. Common mistakes + readiness checklist

Common mistakes

- Forgetting a carry when a column sums to 2 or 3.
- Mixing up the unsigned range (0–255) with the signed range (−128 to 127).
- Converting a number fully just to decide whether it is negative. Only the MSB matters.
- Using the carry-out to judge **signed** overflow. Carry-out only tells you about unsigned overflow and borrow.
- Sign-extending with 0s instead of copying the sign bit.
- Writing `subi` or `blt` when they are not on the allowable list.
- Forgetting ×4 for word offsets (or wrongly applying ×4 to byte/char arrays).
- Loop tests that branch out on the **true** condition, which inverts the loop.
- Evaluating both sides of `&&` / `||` instead of short-circuiting.

Readiness checklist

- [ ] Blind pass over all five section IDs
- [ ] Every Group A item answered correctly from memory (the biggest Test 1 loss area)
- [ ] Two subtraction problems done both ways (borrow method and invert-add-1)
- [ ] GS-D4 written from scratch with no notes
- [ ] Full-adder table and equations redrawn from memory
- [ ] One 25-minute timed paper mock (Mon Sep 28)
- [ ] Logistics: Oct 1 in lecture, sleep, no new topics Wednesday night

---

7. Day-by-day prep plan (Tue/Thu light; no day-before cram)

Day	Focus	Section IDs	Notes
Thu Sep 24 evening	Triage; binary refresh	`252-T2-binary`, intro	Short
Fri Sep 25	2's complement, subtraction, Group A and C	`252-T2-binary`, `252-T2-gates`	Deep day
Sat Sep 26	MIPS intro, loops, arrays; Group D	`252-T2-mips-intro`, `252-T2-mips-loops`	Deep day
Sun Sep 27	Log mistakes, then retake	All	Blind, check, retake
Mon Sep 28	**25-minute timed paper mock** (§4b)	All	Then review misses
Tue Sep 29	LIGHT flash only	Rotate	Class day; HW2 due 7:00 PM
Wed Sep 30	LIGHT active recall only	All	**No new topics**; Asm 2 due (aim for 5:00 PM)
Thu Oct 1 morning	Calm buffer	—	Test in lecture

---

8. Changes in this review (2026-09-24)

- Changed all times from "PT" to Arizona time. The context file says America/Phoenix. Clock times match Pacific until Nov 1.
- Added nearby deadlines from the inventory, including the Asm 2 5:00 PM vs 7:00 PM conflict (prefer Gradescope; finish by 5:00 PM to be safe).
- Completed the full-adder truth table, which previously had four missing rows.
- Removed LaTeX markup that printed literally in the PDF.
- Added explanations of subtraction, unsigned vs signed overflow, "negative or not", approximations, and why there is no `subi`.
- Added practice items GS-A4–A7, B4–B6, C4–C5, and D4–D6, plus a 25-minute timed mock. These target your own Test 1 and Sim 1 feedback themes, and all have answer keys.
- Numeric grades are described in words only, because this review copy lives in a public repository.
- Noted Deck 05 (ALU) as linked that week but **not** claimed as Test 2 scope.

9. Study-aid / academic-integrity disclaimer

Study aid only — not for submitted work. Do not submit this guide, its answers, or AI-generated solutions as coursework or test answers. Sim projects prohibit AI help, so nothing here targets Sim 3. Follow CSC 252 and University of Arizona academic-integrity and AI-use policies. Coverage stays **provisional** until the instructor publishes the official scope.